PCASH Security Audit
Proof-Relation Soundness

Auditician
GPT-6 Luna, maximum effort, fast service tier

November 2, 2026
Audited repository: `pcash-protocol/pcash`
Commit: `51b1826dff701e8c471dd066c2197130153e68e2`

Contents

1 Introduction **1**
1.1 Scope . . . 1

2 Threat Model **1**
2.1 Adversaries and assumptions . . . 1
2.2 Security invariants . . . 2
2.3 Known limits . . . 4
2.4 Deferred properties . . . 4

3 Security & Privacy Findings **4**
3.1 Issue 1: Sponsorship comments misdescribe contract creation . . . 4

4 Recommendations **5**

5 Future Work **5**

6 Conclusion **5**

7 Appendix **5**
7.1 Audit coverage and test evidence . . . 5
7.2 Audit references . . . 6

1 Introduction

This report presents a soundness-focused security audit of the PCASH repository `pcash-protocol/pcash` at commit `51b1826dff701e8c471dd066c2197130153e68e2`. PCASH derives a private-value chain from Ethereum history. Transactions carry proofs that combine account rules, event applications, protocol kernels, and hiding kernels; nodes reconstruct the public statement, verify the proof against release-pinned material, and apply the resulting state transition.

AI notice. This security audit and report were generated by AI. Bugs may have been missed; this report is not a security certification.

The engagement asked whether an accepted transaction could create value, spend or redirect another account's notes, or carry an effect its proofs and signatures do not authorize. The in-scope set comprised 202 tracked files listed by the audit packet, covering protocol and reference-rule circuits, recursive composition, signature and hash circuits, release pins and generators, node verification and derivation, canonical state, Ethereum data handling, and issuance. The audit completed file-by-file review, 38 global focus reviews, and five proof-driven reviews covering ten user-facing properties.

The audit confirmed one *Info* documentation defect: two comments incorrectly describe contract creation as sponsored by a user. No runtime proof-soundness violation was confirmed. The issue does not alter current transaction behavior. It is described in Section 3.

Verification used the target commit and prepared bundle. Nargo checks counted as evidence used the pinned Nargo/noirc `1.0.0-rc.3` toolchain; the pinned Barretenberg release was `6.0.0`. Rust checks used Cargo/rustc `1.94.1`; JavaScript checks used Node `26.5.0`. The full release check and the S and L native-composition suites passed. Key results are summarized in Appendix 7.

1.1 Scope

The review covered all 202 tracked files selected by the engagement inventory. Tests, fixtures, vectors, and benches were used as evidence and were not separate local-review targets. The audit performed 38 global focus reviews, with at least five reviews in each required category and all eight packet-specified focus areas represented.

The audit did not certify the SDK and wallet privacy/recovery paths, RPC/P2P and other service paths, marketplace, token or lineage applications, other hosts, general availability, or the separate `derivation-proof/` subsystem. The packet lists 45 unmatched tracked paths, primarily the derivation-proof tree and `deny.toml`; these were classified as unreviewed, not passed. The auditors also read internal documents omitted from the public tree. The public commit's tree differs from the audit target in two documentation sentences updated by a later documentation fix, one in `docs/guides/building-on-pcash.md` and one in `web/src/routes/docs/LearnPage.tsx`; the code is unchanged. Deferred code was read only when needed to trace an in-scope dependency.

2 Threat Model

This section summarizes the final Invariant-Centric Threat Model. The model describes expected properties, not results that are automatically established by source review. Its detailed, stable invariant IDs are retained.

2.1 Adversaries and assumptions

The primary adversary is an Ethereum sender who can post arbitrary Inbox bytes, blobs, logs, and proof bytes, or deploy a contract that emits a `PcashBatch` event. Other relevant adversaries are malicious rule/event authors, counterparties, block builders, and relayers. A malicious custom rule can endanger accounts that install it, but must not weaken the shared ledger relation for other accounts.

The audit assumes Ethereum consensus and historical data availability, and trusts the node's configured execution RPC for Ethereum truth (T1–T2). It trusts the cryptographic assumptions of the pinned CHONK construction and setup, but does not assume the PCASH circuits, recursive composition, verifier integration, or release checks are correct (T3–T5). Ordinary reference use assumes the user's device and signer display are not compromised; the tagged `K-APP` scenario treats the host and prover as malicious while retaining the separate signer and its display (T6). Approval replay between networks deliberately sharing a chain ID is outside the model (T7).

2.2 Security invariants

Value and settlement

V1 – No counterfeit value. Native PCASH is created only by the specified issuance process; transaction inputs cover outputs and fees.

V2 – Supply cap. Issuance respects the global cap, per-block allowance, equal per-action reward, and exact integer rounding.

V3 – No double spend. A note is consumed at most once in canonical order; an invalid transaction does not consume its nullifiers.

V4 – Fixed-supply assets. A standard asset is created once under its definition and cannot become native value or another asset through the protocol relation.

V5 – Failed means no effects. A failed admitted transaction consumes no nullifiers, creates no outputs or account writes, pays no fee, and earns no issuance; its failure result is still recorded.

V6 – Fees move existing value. Open and selected fees move existing PCASH; a transaction's later issuance cannot fund itself.

V7 – Same state after recovery. Re-deriving the same canonical Ethereum history after supported restart, crash, or reorg yields the same consensus state and results.

Account authority and approvals

A1 – Only account rules authorize action. Spending and administration require the membership specified for the action; administration uses the live row. All-padding mining spends no account funds.

A2 – The user gets the signed effects. A reference approval binds the effects it describes; a prover cannot alter those effects after signing. Unsigned delivery choices and event claims are not automatically signer-approved.

A3 – Approvals do not replay. An approval is limited to its account, role, chain, nullifier context, and signed time window.

A4 – Account address belongs to its creator. User-account creation requires the corresponding direct Inbox sender or valid signature; existing accounts cannot be replaced or re-registered, and the nullifier-key commitment is immutable.

A5 – Rule changes use current state. An update replaces the exact live row, including earlier successful same-block updates; a removed administration rule cannot use a historical root to reinstall itself.

A6 – Proving software cannot spend alone. Under K-APP, the host cannot authorize an unsigned payment without the separate signing key or add unauthorized outflows to a signed payment, except for the documented relay-fee destination limit K61.

A8 – Others cannot forge consumption. An unrelated party cannot invalidate another user's notes or consume an account-scoped marker without the required authority.

A9 – No discretionary gatekeeper. Any party may deliver an otherwise-valid transaction through an allowed carrier, subject to inclusion and transaction conditions.

R2 – Change is recomputable. Reference rules force owner-derived change and a deterministic amount split that the owner can recompute from retained context.

Proof, state, and delivery bindings

D1 – Installed rule is the gate. A spend uses the correct account, rule, configuration, and membership proof; administration uses live state.

D2 – Action commits to its effects. Action and private-value commitments cover role, chain, profile, fees, validity, slot classes, note values, role-specific extra data, and one delivery commitment over ordered output-data hashes and the arena.

D3 – Weak rules cannot break the ledger. A malicious installed rule can endanger its own account but cannot counterfeit value, spend another account's notes, or bypass canonical nullifier checks.

D4 – Contract identity is complete. A contract address commits to its full immutable function set and related account material.

D5 – Time limits are enforced. Canonical Ethereum block time determines whether a transaction is before its start or after its expiry.

D6 – Opened Ethereum facts are authentic. A proof opening binds normalized logs to emitter, exact topics/data, transaction/log position, and canonical L1 origin without lossy hash reduction.

D7 – One-time markers are scoped. Replay markers are bound to their authorized account and entry; a free witness alone does not prove one-time use.

D8 – Events bind to every rule and the action. Application identity, event claim, action, anchor, and entry agree across the composition and approving rules. The target has four proof-bearing relation classes: transfer, sponsored transfer, asset creation, and user-account update; the first three have S/L wrappers and account update has an S wrapper. Type-0x02 creation is proof-free; attached creation rides a type-0x00 action.

D9 – Sponsorship is bounded. A sponsor's contribution is limited to the approved native fee and relay outflow for the named contract.

D10 – Types and profiles stay separate. A proof or approval cannot be reinterpreted under another type, profile, statement layout, or hiding key.

D11 – Rule privacy. User rule/configuration openings are not additional public statement fields, apart from deliberate disclosure and public contract definitions.

D12 – Shared encodings agree. Rust, Noir, and TypeScript agree on layouts, hashes, field order, commitments, and signed messages against the specification and independent vectors.

D13 – Complete proof and release binding. The composed relation, actual verification keys, allowlists, hiding statement, node-reconstructed statement, and recorded statement hash are bound end to end.

D14 – Historical results are not misrepresented. A result opening binds to an authenticated history root, correct block results root/counts/partition/index and leaf; consumers additionally check success and the expected type, funding, profile, statement, and output position.

D15 – Carrier condition is exact. A nonzero condition permits only the named carrier kind, sender or emitter, and exact context; Inbox and event carriers remain distinct, and mismatch fails before state consumption.

Node agreement and release integrity

L1 – Correct settlement beneficiary. Fees and issuance settle to the specified batch owner without duplicate rewards.

N1 – Same inputs, same result. The same release and canonical Ethereum history produce the same ordered outcomes, state, and result roots regardless of cache/prewarm order or machine load.

N10 – Verifier and blob integrity. Incorrect verifier/setup files are refused; blob bytes are checked against canonical Ethereum versioned hashes, and unavailable data is not called invalid.

2.3 Known limits

The model does not promise a light client (K3), historical-blob availability (K5), a separate per-block proof-work budget or uninterrupted operation after infrastructure failures (K6–K7), immediate revocation of historical spending authority (K9), human-readable review of a new rule-set root (K10), decryptable delivery (K11), safe custom rules or automatic gate support (K38–K39), signer approval of unsigned event claims (K40), account membership for all-padding transfers (K41), direct chain-ID binding in every nullifier (K44), safety of every application rule (K46), entropy of prover-chosen values (K51), or relayer delivery for carrier-bound transactions (K60). K61 states that the reference approval fixes a selected relay-fee amount and opaque owner commitment, not a human-readable relayer destination. These limits do not excuse a violation of the narrower invariants.

2.4 Deferred properties

Privacy P1–P9; single-key separation A7; most recovery R1 and R3–R5; malicious-node wallet properties W1–W8; quote and wallet behavior Q1–Q4; node availability and public-service properties N2–N9 and N11–N12; marketplace B1–B7 and S1–S7; token C1–C6; lineage G1–G6; and Ethereum submission L2–L6 remain unreviewed/out of scope.

3 Security & Privacy Findings

The audit confirmed one issue. No confirmed Critical, High, Medium, or Low runtime security vulnerability was found in the reviewed soundness scope.

3.1 Issue 1: Sponsorship comments misdescribe contract creation

Severity: *Info*

Location:

- `circuits/protocol/lib/src/ledger/fee_payment.nr:1-10`
- `circuits/protocol/lib/src/authorization/statement.nr:104-105`
- Related call sites in `circuits/protocol/lib/src/kernels/sponsored_transfer.nr`

The comments describe contract creation as an action for which a user can pay a sponsored contract fee. They also say that the related fee section omits issuance. The actual `native_fee_payment` relation is used by the type `0x03` account-update kernel. A type `0x00` sponsored transfer uses shared conservation and change helpers, retains the type `0x00` issuance fields, and authenticates an existing contract account for its asset action. Standalone type `0x02` creation is unsigned and proof-free, pays no PCASH, and earns no issuance. An attached creation shares the carrying type `0x00` action's fee and outcome.

The two inaccurate comments are:

Listing 1: Comments in fee_payment.nr (lines 1-10) and authorization/statement.nr (lines 104-105)

```
// This is the value relation of every fee-bearing account action.
// A user account update funds its own fee this way, and a sponsored
// contract-account creation is funded this way by the sponsor.

// A user account pays the native fee for one named contract: its asset action
// in a sponsored transfer, or its creation.
```

Impact. No attacker-triggerable runtime effect was found. The comments are a misleading code map that can misdirect later reviews or changes to sponsor authorization and issuance behavior. This is an informational documentation defect, not a runtime security vulnerability.

Recommendation. Describe `native_fee_payment` as the type `0x03` account-update relation, distinguish its shared helpers from the type `0x00` sponsored-transfer relation, and describe the contract-fee-sponsorship action as paying for an existing contract's sponsored asset action, not its creation.

Validation. The only production call to `native_fee_payment` is the account-update kernel. The only production call to `contract_fee_sponsorship_action_commitment` is in the sponsored-transfer kernel after the primary contract account is authenticated. The proposed documentation-only corrections passed `git apply --check`; no runtime test applies.

4 Recommendations

1. Correct both sponsorship comments described in Issue 3.1.
2. Correct the supplied navigation survey's public-input formula. The target protocol specification and implementation use `I + O + 11`; the S and L statements contain 20 and 75 public inputs respectively. This is a packet-document typo, not a product-code finding.
3. Keep the independent issuance oracle, S/L native-composition controls, event-linkage substitutions, and release-check KATs in the recurring verification workflow. Source-level assertions alone do not establish the compiled recursive relation.

5 Future Work

Future reviews should cover the deferred wallet privacy, key separation, backup recovery, malicious-node interaction, RPC/P2P service boundaries, marketplace, token and lineage applications, and the separate derivation-proof subsystem. They should also cover denial of service and availability, which this soundness-only engagement excluded absent acceptance divergence. The 45 unmatched tracked paths and all paths selected by `scope/out.txt` remain unreviewed rather than passed.

This audit applies only to commit `51b1826dff701e8c471dd066c2197130153e68e2`. Changes to circuits, generated constants, manifests, pins, toolchain releases, or protocol statement layouts require renewed verification against the actual compiled bundle.

6 Conclusion

The audit found no runtime proof-soundness violation in the reviewed scope under the stated Ethereum, execution-RPC, proof-system, and release assumptions. The sole confirmed finding is an Info documentation defect about sponsorship and contract creation. This conclusion is not a proof that all invariants hold, and it does not certify deferred or unmatched code.

7 Appendix

7.1 Audit coverage and test evidence

The audit completed local review of all 202 scoped files, 38 global focus areas, and five proof-driven groups covering ten properties. Tests and generated-artifact checks ran in prepared scratch worktrees at the target commit; scratch-only test variants were preserved and not applied to the target.

Key pinned-toolchain results include:

- Protocol library: 210 tests passed; shared reference-rule library: 94; signature library: 11.

- Rust crypto: 19 tests; state: 109; derive: 216; L1: 37 unit and 13 integration tests; DA: 12 default-feature and 13 KZG-feature tests. One separate Sepolia-only L1 test was not run because it requires a live network.
- Verifier: 41 regular tests and 12 tests ignored by default passed with the pinned binary/CRS, including startup known-answer checks and malformed-proof fuzzing. A rustdoc temp-directory failure on the default path was resolved by running with an allowed temporary directory; no source change was made.
- Native recursive composition: S passed 27/27 and a scratch-only L variant with shape 1 and L links passed 27/27. Event linkage passed 26/26 existing cases and 12/12 added scratch-only cases across asset-creation S/L and account-update S.
- Full release check passed, including eight positive/negative native CHONK known-answer cases. Browser asset, bundle, profile, reference-message, and release-pin checks also passed.
- Independent issuance verification matched 30 vectors and all 8,519,681 Q16 exponents. An independent spec-driven Python implementation matched the checked encoding vectors, including domain tags, commitments, carrier contexts, state/header roots, and result leaves.

All reported Nargo evidence used the pinned Nargo/noirc `1.0.0-rc.3`; pinned Barretenberg was `6.0.0`. Ambient Nargo beta.22 output was excluded. Cargo/rustc were `1.94.1`, and Node was `26.5.0`.

7.2 Audit references

The protocol specification is the normative source for protocol requirements [1]. The audit threat model is user-facing context for the invariants and accepted limits [2].

References

[1] Auditician. PCASH Protocol Specification, 2026. Normative protocol specification for the audited PCASH commit.

[2] Auditician. PCASH Invariant-Centric Threat Model, 2026. Threat-model context for the audited PCASH commit.